Università
Ca'Foscari
Venezia

[CM0468] CLOUD COMPUTING AND DISTRIBUTED SYSTEMS (CM9)

Seminar presentation

NETFLIX system design and sw architecture

Author:
Santello Veronica, 870320

Academic Year:
2020/2021

Date:
27/4/2021

Contents

1 Introduction

Netflix is a US company operating in the distribution via Internet of movies, television series and other entertainment content for a small price. It counts about **200+ million of subscribers**, **200+ countries**, **2879 video contents** and **125 million viewing hours per day**.

This video streaming platform has been depopulating in the last few years almost all over the world and therefore it is interesting to find out how its internal design is implemented in order to offer a high quality service.

Figure 1: Netflix logo[1]

The main idea is to understand how this particular distributed system works, in its internal mechanisms, in order to offer:

- high level of **transparency** to the clients
- high **resolution** of video for every type of device
- high **response time**
- high **reliability**

The implementation of the Netflix design and its software architecture is an interesting case of study that allow us to understand how this distributed system, that we use every day, works and interconnects two famous clouds (Open Connect & AWS) in order to provide a **high quality film streaming service**.

The Netflix SW interface is written in `React js`, that is *"an open-source, front end, JavaScript library"*[7], with three main positive features: startup speed, run-time performance and modularity.

2 Components involved

We can identify three main components that are involved in the Netflix system:

1. **Clients**: all devices (User Interfaces) which are used to browse and play Netflix videos. Possible devices are for example Smart TV, XBOX, laptop, I-pad smartphone etc. Netflix checks its performance via the provided SDK which must be installed on the device.
   An SDK, i.e. software development kit, generically indicates a set of tools for software development and documentation. The specific Netflix's SDK is called **Netflix Ready Device Platform (NRDP)**.

2. **Open Connect (or Netflix CDN)**: CDN (i.e. Content delivery network) is the network of distributed servers in different geographical locations and Open Connect is the personalized Netflix global CDN.
   It handles everything which involves video streaming. It is distributed in different locations over the world and once you hit the play button the video stream from this component is displayed on your device. Obviously, the video will be served from the nearest server instead of the original one,in order to improve the response time.
   Netflix collaborates with about 1000 ISP in order to offer a good video streaming service minimizing the distribution of traffic with the help of OCA (i.e. Open Connect Appliances) devices. They ensures that a considerable amount of Netflix traffic is offloaded from peering or transport circuits (usually an ISP handles about 90% of the Netflix's traffic).
   So, for example in the figure 2, if a user made a request, it isn't answered directly to the nearest Netflix data center, but by the nearest OCA located on the nearest data center of its own ISP. In other words, an OCA, **is a mini-server of Netflix located in many different points in order to improve the bandwidth in a certain location.**

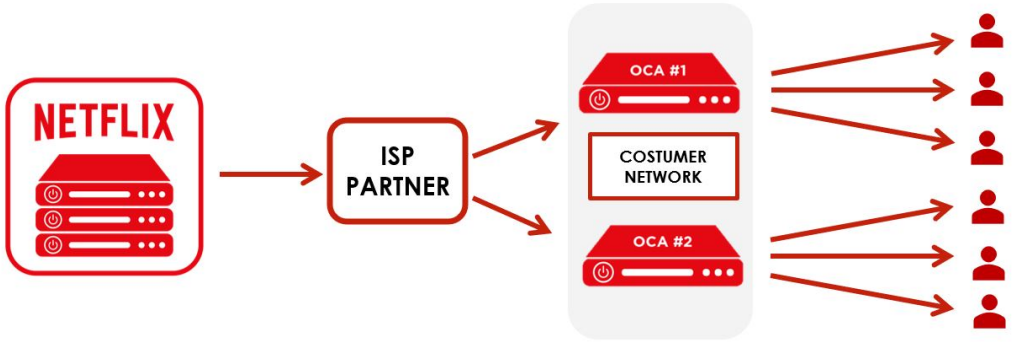

Figure 2: Open Connected (Netflix CDN)[5]

3. **Back-end**: deals with anything that doesn't involves video streaming, such as login, recommendations, search, user history, the home page, billing, customer support, etc. The requests are take care by the cloud AWS (i.e. Amazon Web Services).

Some of the components of Back-end with their corresponding AWS services are listed as follows:

1. Scalable computing instances (AWS EC2)
2. Scalable storage (AWS S3)
3. Scalable distributed databases (AWS DynamoDB, Cassandra)
4. Video processing and transcoding (purpose-built tools by Netflix) [11]

The following diagram 3 illustrates how the **playback process**, that is the direct communications between the client and the two clouds, works:

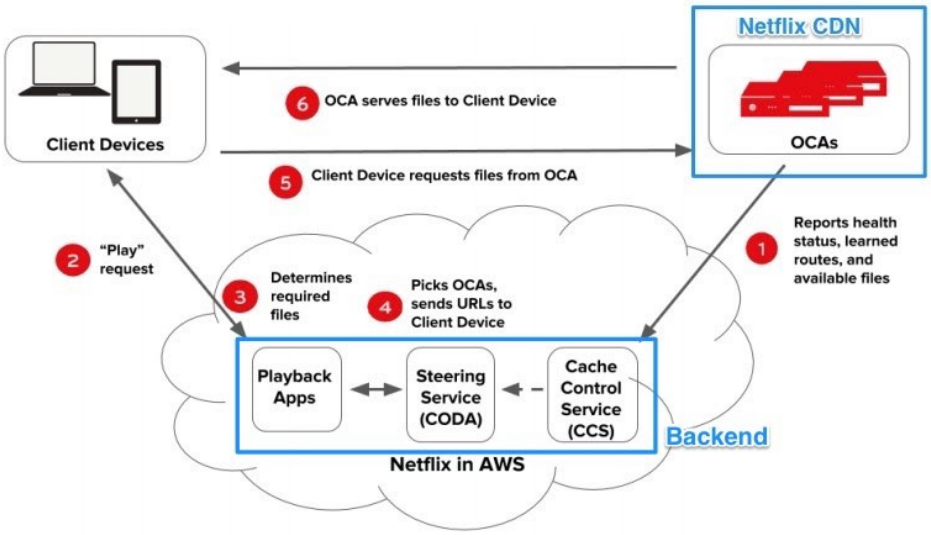

Figure 3: Playback architecture for streaming videos[11]

1. OCAs constantly **send health reports** about their workload status, rout-ability and available videos to Cache Control service running in AWS EC2 in order for Playback Apps to update the latest healthy OCAs to clients.
2. A Play request is sent from the client device to Netflix's Playback Apps service running on AWS EC2 to **get URLs for streaming videos**.
3. Playback Apps service **must determine that Play request would be valid in order to view the particular video**. Such validations would check subscriber's plan, licensing of

the video in different countries, etc.

4. Playback Apps service talks to Steering service also running in AWS EC2 to **get the list of appropriate OCAs servers of the requested video**. Steering service **uses the client's IP address and ISPs information to identify a set of suitable OCAs work best for that client.**
5. From the list of 10 different OCAs servers returned by Playback Apps service, the client tests the quality of network connections to these OCAs and **selects the fastest, most reliable OCA to request video files for streaming**.
6. The selected OCA server **accepts requests from the client and starts streaming videos**. [11]

In the above diagram, Playback Apps service, Steering service and Cache Control service run entirely in AWS cloud based on a micro-services architecture.

2.1 Protocols involved

Client apps uses **NTBA protocol** for Playback requests to ensure **more security over its OCA servers** locations and to **remove the latency** caused by a SSL/TLS handshake for new connections.

TSL and its predecessor **SSL** are cryptographic protocols that provide authentication, data integrity and confidentiality by operating above the transport layer.
Currently, SSL/TLS protocol has been replaced with **Message security layer (MSL)** after demonstrating security weaknesses.

Another specific protocol, this time involved into micro-services communication, is **REST (REpresentational State Transfer)** or **gRPC** that allows RPC on a different machines .

gRPC is based around the idea of defining a service, specifying the methods that can be called remotely with their parameters and return types as they are local.
On the server side, the server implements this interface and runs a gRPC server to handle client calls. On the client side, the client has a stub (referred to as just a client in some languages) that provides the same methods as the server.

3 Global system design

On the following figure 4 is shown the internal global system design of Netflix. This schema is quite complicated since there are several clouds, software, databases and devices that collaborate in order to improve the quality of Netflix services.

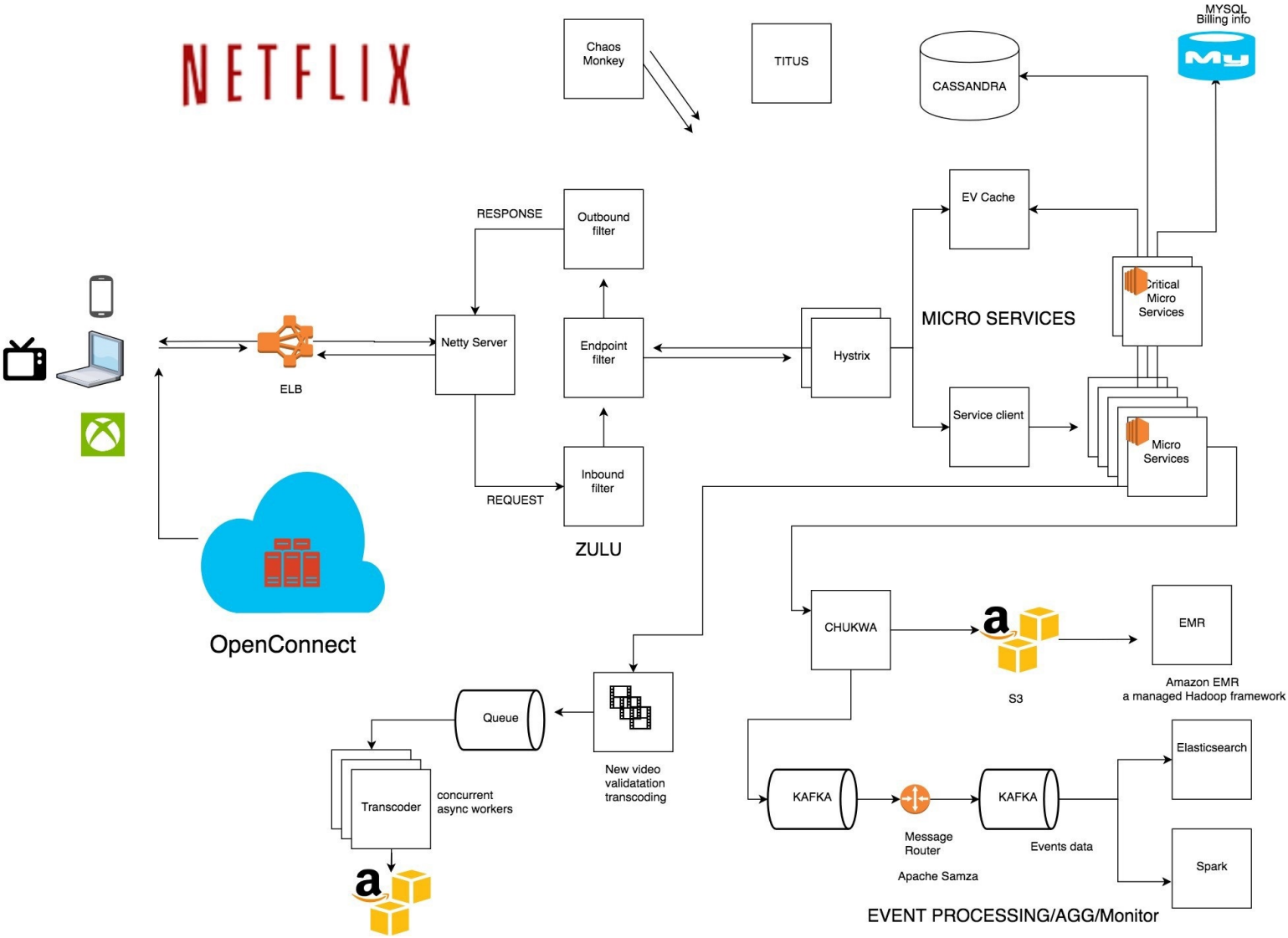

Figure 4: Netflix system design[2]

In this relation, I want focus the attention into 4 main part of this global system, that are:

1. **Trans-coding process**, that is how a new film is inserted and distributed on the platform. In this point I want focus the attention on the `Transcoder` entity of AWS (bottom left in the figure 4) that, after receiving a new movie, computes different versions of it and push them into all the open connect servers.
2. The **elastic load balancer**, that is a distribution of client's requests into the most suitable AWS server. The component affected (`ELB`) is immediately next to the client in the figure 4.
3. Use of **ZUUL** and **Hystrix** that provide respectively a gateway service and the management of the latency. ZUUL is composed of four entities as you can see in the figure 4: `server, inbound filter, endpoint filter, outbound filter`.
   Hystrix entity is represented on the right of ZUUL.
4. Finally, the **micro-services architecture** that is composed of the `EV-cache`, the `service client` and a set of `Critical micro-services` and a set of `normal micro-services` interconnected.

3.1 Trans-coding process

This section aswer the question: *How Netflix insert a new Movie/Video?*

Netflix supports more than 2200 devices and each one of them requires specific resolution and format. To make the videos accessible on different devices Netflix performs the trans-coding process, which involves **finding errors** and **converting the original video into different formats and resolutions**.

Netflix also creates file optimization for different network speeds, and obviously, the higher is the network speed, the better will be the video quality.

It's possible that, when the bandwidth is less, Netflix continuously changes resolution according to it and then, from one moment to the next, you switch from seeing the film in high resolution to a low one (**bit-rate streaming**). To do that, approximately, Netflix creates **1200 copies of each movie**

Suppose that Netflix wants to insert a new film which weighs 50 GB; if there was only that copy, it would be difficult to stream it to every customer and on any device.

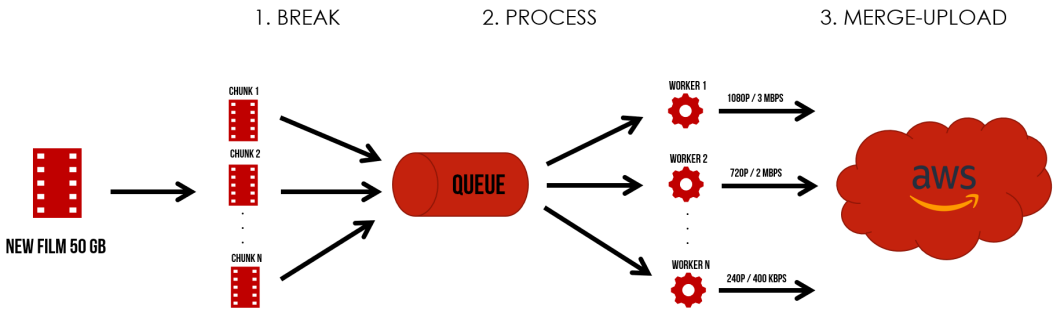

Figure 5: Trans-coding process[8]

As we can see on the figure 5, there are 3 different steps:

1. **Break**: divide the film into many chunks, usually 3 minutes of film, and insert them into the queue. After this point, the `transcoder` entity takes control.
   Latest SW versions of the transcoder have an **important change**: the chunks are no longer than 3 minutes but have a finer granularity (4 seconds) and are called shots. These shots are then collected in scenes (of different lengths) in order to improve the visual experience of users watching action movies.

   When a film is requested by a user, not from the beginning but from a specific point, OC transfers the entire scene in which the user's request falls and allows continuous viewing without changes in resolution.

2. **Process**: workers in AWS `transcoder` convert these chunks into different formats and resolutions. A single chunk is converted into a couple *[format, resolution]* this means that, if F is the number of all possible formats and R is the number of all possible resolutions, each chunk has $F \cdot R$ versions.

3. **Merge-Upload**: When new video files have been trans-coded successfully and stored on AWS S3, the **control plane services** on AWS will transfer these files to OCAs servers on IXP sites (list of Internet exchange points). These OCAs servers will apply **cache fill process** to transfer these files to OCAs servers on ISPs sites under their sub networks.

After this procedure, when a client hit the play button,the application find the best open connect server that starts to stream the video to the client device. Client devices applications are very intelligent, since they constantly check the best OCA in every moment during the streaming. This guarantees no interruptions and a good viewing experience. **All this could be achieved because the Netflix Platform SDK on Client keeps tracking the latest healthy OCAs retrieved from Playback Apps service**.

Anything not related to streaming, for example the movies you have seen, searched etc., will be saved into AWS S3 that uses **machine learning algorithms** to make best suggestions and recommendations to the user.

3.2 Elastic load balancer

A famous problem that affects many distributed systems, is the distribution of clients requests in a balanced way between servers.

ELB in Netflix is responsible for routing the traffic to front-end AWS services. ELB performs a two-tier load-balancing scheme where the load is balanced over zones first and then instances (servers).
As we can see on the figure 6 below, the first tier consists on basic DNS based Round Robin Balancing. So when requests lands on the first ELB, they are distributed across different zones. A **zone** is simply a logical group of servers, for example in the USA there are 3 zone as in the image 6.

The second tier is an array of load balancer instances and it performs the **Round Robin scheduling** technique to distribute the request across the instances that are behind it in the same zone.

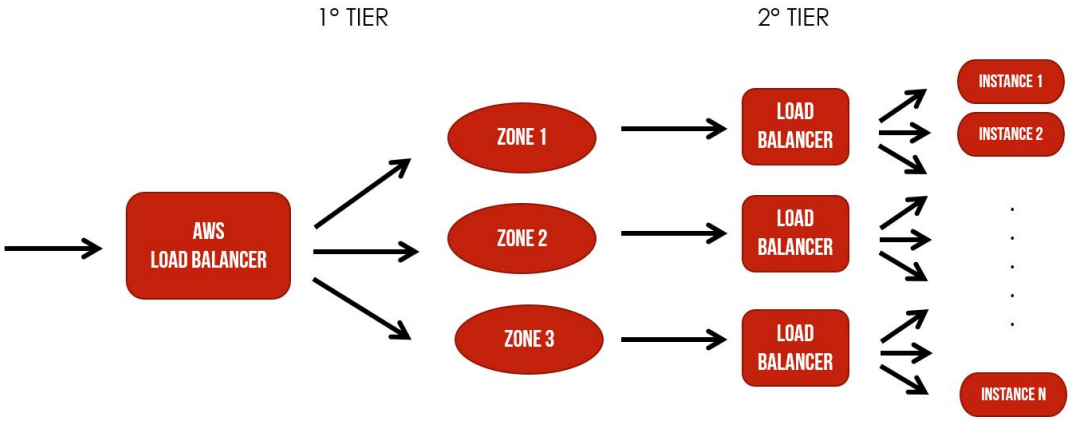

Figure 6: Elastic Load Balancer[4]

3.3 ZUUL and Hystrix

ZUUL is a **gateway service** that provides different features as dynamic routing, monitoring, resiliency, and security. The idea behind ZUUL is to provide easy routing of clients requests based on query params, URL, path.
As you can see in the image 7 below, there are different parts that are involved in this process.

Let's understand them one by one:

- **Netty server**: takes the responsibility to handle the network protocol, web server, connection management, and proxying work. When the request will arrive on the Netty server, it will proxy the request to the inbound filter.
- **Inbound filter**: is responsible for authentication, routing, or decorating the request. Then it forwards the request to the endpoint filter.
- **Endpoint filter**: its works depends on the type of request. It can return a static response or to forward the request to the back-end service. Once it receives the response from the back-end service, it sends the response to the outbound filter.
- **Outbound filter**: is used for zipping the content, to calculate the metrics, or adding/removing custom headers. After that, the response is sent back to the Netty server and then it is received by the client [4].

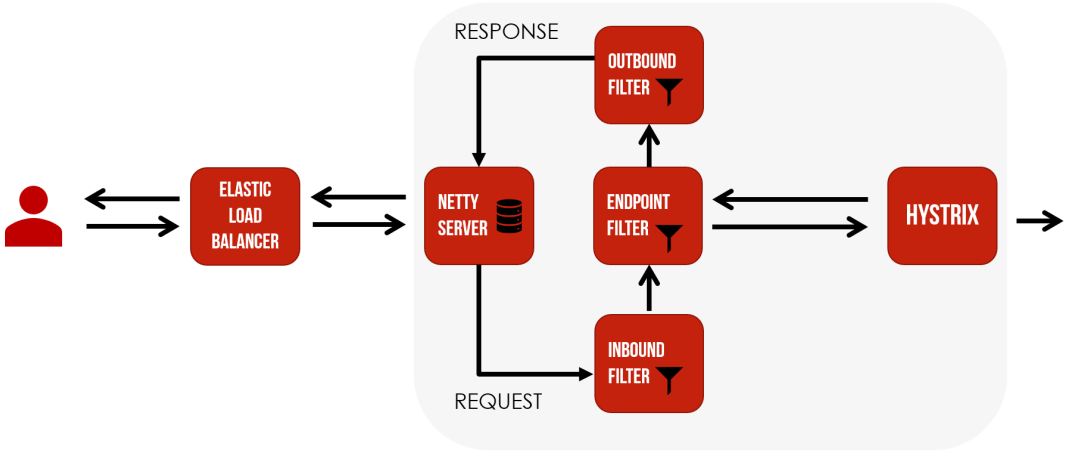

Figure 7: Relation between ZUUL and Hystrix[4]

Now the question is, *what are the benefits of having a gateway service like this?*

The main advantages are the possibility of **distribute different requests** into different types of servers. Also, is possible to do **load testing** on specific server in real-time. The latter is very useful for the developers. We can also **filter the bad request** by setting the custom rules at the endpoint filter.

Now let's see what is Hystrix. Hystrix is a latency tolerance and fault tolerant design library. In a complex distributed system a server may rely on the response of another server. Dependencies among these servers can create latency and the entire system may stop working or crash if one

of the servers will fail at some point.
To avoid this problem Hystrix isolate the host application from these external failures. Let's see an example of a typical issue that Hystrix solve.

Suppose that several requests arrives at the endpoint, it might propagate the requests into other different servers. An example of success response is illustrated into the figure 8 where the request1 is transferred from the endpoint to the micro-service 1 into the server1, and the latter response successfully.

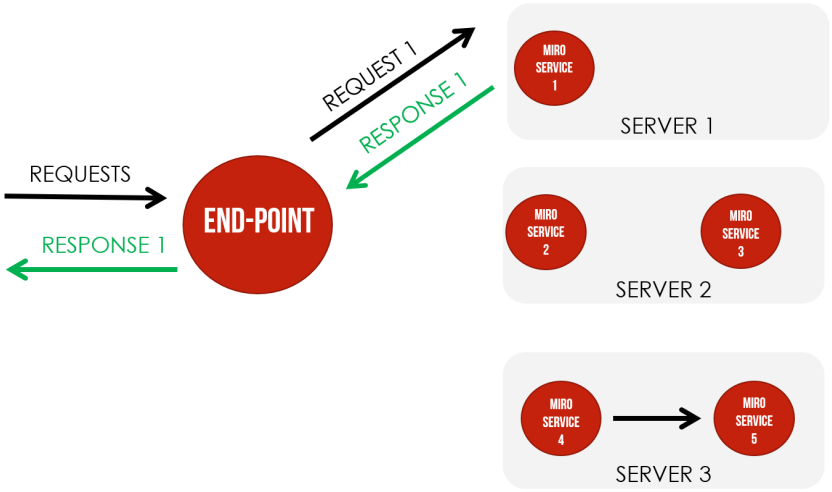

Figure 8: Successful communication between endpoint and another server[8]

The problem arises when, for instance, a micro-service crash and so the endpoint might suffer a lot of latency propagated by a chain reaction. An example is shown into the figure 9.

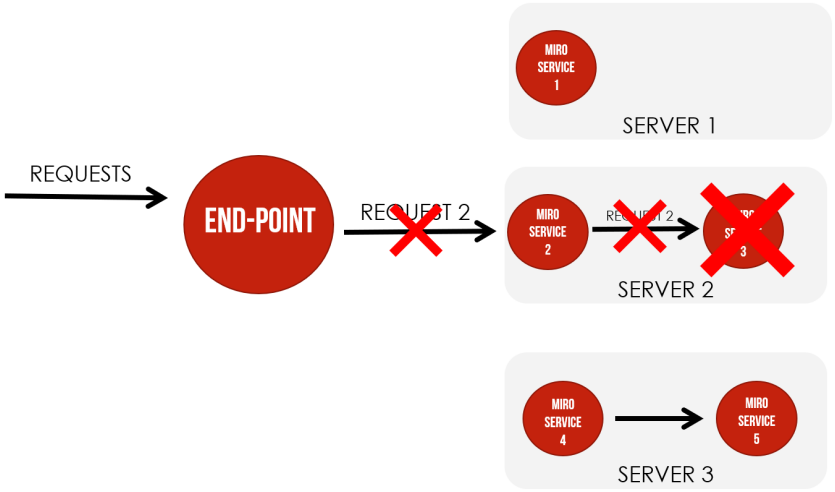

Figure 9: Example of cascading failures[8]

In this last example, the request2 arrived at the endpoint, it is forwarded to the micro-service 2 (into the sever 2) and so is still forwarded to the micro-service 3 (always into the server 2) but the latter produce an error, and so the final response could go in error.

Finally, we can summarize the benefits of Hystrix as following:

1. **Stop cascading failures**
2. **Control over latency and failure** from dependencies accessed
3. Fail fast and **rapidly recover**
4. **Fallback and gracefully degrade** when possible
5. **Real-time monitoring, alerting, and operational control** [4].

3.4 Micro-services architecture

This subsection will answer the question: *How to make the Netflix architecture reliable and faster?*

First of all, micro-services architecture refers to a technique that gives modern developers a way to design highly scalable, flexible applications by decomposing the application into discrete services that implement specific features. These services, often referred to as *"loosely coupled,"* can then be built, deployed, and scaled independently [9].

Netflix's architectural style is built as a collection of services. When the request arrives at the endpoint it calls the other micro-services for required data and these micro-services can also request for the data to different micro-services and so on and so forth. The protocol used into communication between micro-services is **REST** (which uses a subset of HTTP) or **gRPC**.

Another important component is the **Application API** that plays a role of an orchestration layer to the Netflix micro-services. The Application APIs are defined under three categories: **Sign-up API** for non-member requests such as sign-up, billing, free trial, etc., **Discovery API** for search, recommendation requests and **Play API** for streaming, view licensing requests. A detailed structural component diagram of Application API is provided in the following diagram on th figure10.

Recently, the network **protocol used** between Play API and micro-services is **gRPC/HTTP2** which *"allowed RPC methods and entities to be defined via Protocol Buffers, and client libraries/SDKs automatically generated in a variety of languages"*. [11].

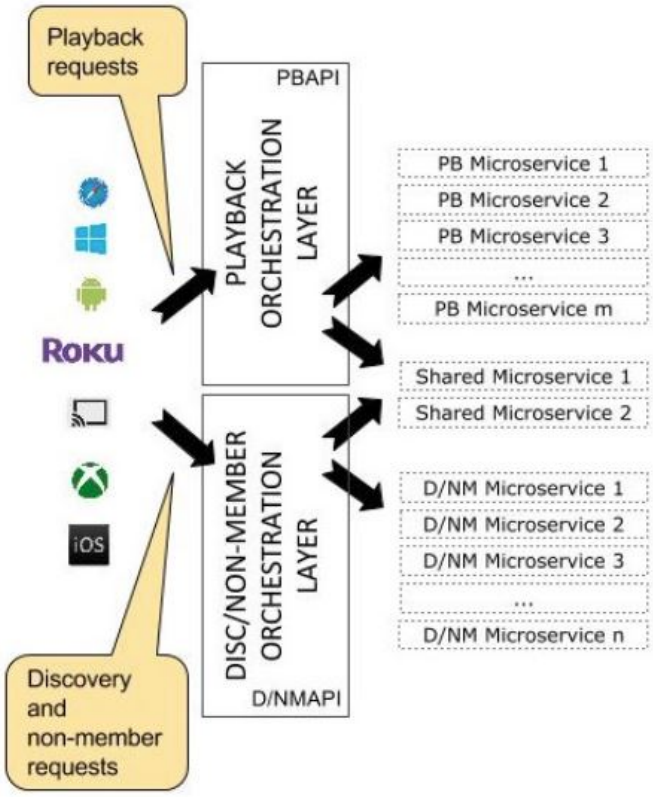

Figure 10: Structure of Application API[11]

The tricks used on this architecture to make a system with this type of architecture **more reliable** are:

- **the use of Hysterix** (Already explained)
- **Separate Critical Micro-services**: it's important to keep critical and non-critical services independent. In this way it is possible to make the endpoints highly available and even in worst case scenarios at least one user will be able to do the basic things.
- **Treat Servers as Stateless**: instead of relying on a specific server and preserving the state in that server, you can route the request to another service instance and you can automatically spin up a new node to replace it. If a server stops working, it will be replaced by another one [4].

For **faster response**, these data can be cached in so many endpoints and it can be fetched from the cache instead of the original server. The problem arise when a endpoint goes down, also the cache goes down, and this can hit the performance of the application.

To solve this problem Netflix has built its own custom caching layer called **EV-cache** which is

based on several instances Memcached.

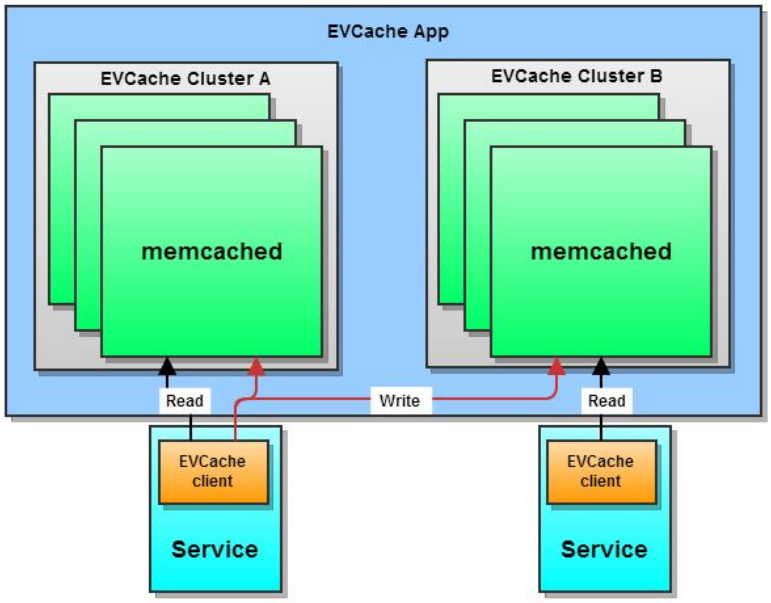

Figure 11: Example of use of EV-cache[10]

In the figure 11 there is a **EV-Cache App**, that is a logical grouping of one or more **Memcached instances** (servers) composed of two **clusters** in the same logical zone.
Instances that can talk memcahced **protocol such as Couchbase, MemcacheDB**.

On this scenario, the data is replicated across both the clusters, in order to improve the availability, and is shared across the 3 instances in each cluster, in order to improve the reliability [10].
All the **reads** by a client are sent to the same cluster instead the **writes** are done on both the clusters. Since the data is always read from the local zone this improves the latency. If an instance failure occurs and some data are lost, now the data can be fetched from the other cluster. The latter process is called **fallback** and is much faster than getting the data from the real source.
This approach increases performance, availability, reliability and handles 30 million request a day and linear scalability with milliseconds latency.

4 Design goals

In this section I would like to deepen the analysis of this design architecture and explain how the most important design goals were achieved:

- Ensure high **availability** for streaming services at global scale.
  In the design of our system, the availability of streaming services depends on both the availability of the back-end services and the OCA servers that store the streaming video files.
  Therefore, its availability depends on several components that involve playback requests: **load balancer** (AWS ELB) that prevent overloading workloads, **API Gateway Service** (ZUUL) that allows dynamic routing, **Play API** that controls the execution of micro-services and prevent cascading failures through Hystrix, **EV-Cache** that replicates data for faster access.

- Tackle network failures and system outages by **resilience**: Designing a cloud system capable of self-recovering from errors or interruptions has always been an important goal of Netflix. Common fixed errors are: dependencies through services, cascading failures to other services, overhead, connection errors to OCAs. Application API uses Hystrix commands to **timeout** calls to micro-services, to stop cascading errors and isolate points of failure from others.

- Minimize streaming **latency**: application API uses Hystrix commands to control how long it wants to wait for the result before getting stale data from the cache. This allows you to control acceptable latency and stop cascading errors for further services.
  If the network connection is unreliable, or the OCA server is overloaded, the client immediately switches to other nearby servers. It can also lower the video quality to match the quality of the network in case it detects degradation in the network connection.

- Support **scalability** upon high request volume: horizontal scaling of EC2 instances on Netflix is provided by AWS **Auto Scaling Service**. This AWS service automatically launches more elastic instances if the volume of requests increases and deactivates unused ones.

5 Conclusion

In conclusion, we can say that we have seen an overview both from the hw and sw point of view of the Netflix distributed system. There is much more to say, especially from the point of view of the DBs used, but this topic is not part of this seminar.

We can conclude by saying that Neflix, like many other large distributed systems, works every day to ensure a quality video streaming service. Each process such as the insertion of a new movie, the management and distribution of requests between servers, the prevention of latency problems, the subdivision of micro-services and the use of innovative caches, allows a high degree of transparency for the user, who from the comfort of his home, watches movies and TV series as if they were local and not in streaming.

References

[1] *Source:* https://www.netflix.com/it/watch-free

[2] *Source:* https://medium.com/@narengowda/netflix-system-design-dbec30fede8d

[3] https://www.youtube.com/watch?v=psQzyFfsUGU

[4] https://www.geeksforgeeks.org/system-design-netflix-a-complete-architecture/

[5] https://www.antoniosavarese.it/2016/03/31/netflix-open-connect/

[6] *Source:*https://candid.technology/how-netflix-work/

[7] *Source:* https://en.wikipedia.org/wiki/React_(JavaScript_library)

[8] *Source:* https://www.youtube.com/watch?v=psQzyFfsUGU&t=492s

[9] https://www.tibco.com/reference-center/what-is-microservices-architecture?utm_medium=cpc&utm_source=google&utm_content=s&utm_campaign=ggl_s_en_uk_TCI_nonbrand_events_beta&utm_term=microservices&_bt=473116235476&_bm=e&_bn=g&gclid=CjwKCAjwjuqDBhAGEiwAdX2cj47Ywmk4bVgpulSj_6aXB1HPpsFcYWHfs7TyQUvDgHiOG_VKyiAZERoCEUIQAvD_BwE

[10] https://netflixtechblog.com/announcing-evcache-distributed-in-memory-datastore-for

[11] https://medium.com/swlh/a-design-analysis-of-cloud-based-microservices-architectur_=_